Python Engineering - Reference Guide

CLASS 16 - Testing & Project Structure

Extended quick-reference guide for basic tests, assertions, numerical tolerance, file organization and execution.

1. test

A test is code whose purpose is to automatically verify that other code behaves as expected.

```
def test_something():
    result = function_under_test()
    assert result == expected
```

2. test_... naming

Names such as test_euler_step clearly identify both purpose and behavior under test.

3. Arrange - Act - Assert

Phase	Question
Arrange	What objects/data/conditions do I prepare?
Act	What function or method do I execute?
Assert	What result do I expect and how do I verify it?

```
# Arrange
state = VerticalState(h=100.0, v=20.0)

# Act
state.euler_step(0.1)

# Assert
assert np.allclose(state.state_vector(), [102.0, 19.019, 0.1])
```

4. assert

```
assert condition
assert condition, 'failure message'
```

True -> continue. False -> Python raises AssertionError.

5. AssertionError

```
assert 2 + 2 == 5, 'Math changed!'
# AssertionError: Math changed!
```

In testing, AssertionError normally means FAIL: execution completed far enough to compare behavior, but expectation was not met.

6. np.allclose()

```
np.allclose(actual, expected)
np.allclose(actual, expected, rtol=..., atol=...)
```

Argument	Meaning
actual	Computed values
expected	Reference values
rtol	Relative tolerance
atol	Absolute tolerance

7. Exact equality vs tolerance

```
actual == expected
np.allclose(actual, expected)
```

Exact equality is often fine for integers/strings. Numerical floating-point results commonly need tolerance.

8. Expected-exception test

```
try:
    operation_that_should_fail()
except ExpectedError:
    pass
else:
    assert False, 'Expected exception was not raised'
```

9. PASS / FAIL / ERROR

State	Interpretation
PASS	Test finishes and all assertions hold
FAIL	Test ends with AssertionError
ERROR	Unexpected exception prevents normal test completion

10. test.__name__

```
print(test_example.__name__)  # test_example
```

Functions are objects; __name__ gives the function name.

11. TESTS list

```
TESTS = [test_initial_state, test_euler_step]
```

Store function references, not function results. Do not add parentheses inside the list.

12. except Exception in the runner

A broad catch is useful at the outer runner boundary so one unexpected test error does not stop remaining tests. We still report type/message rather than hiding it.

13. Path(__file__).resolve().parent.parent

```
base_dir = Path(__file__).resolve().parent.parent
```

From tests/test_file_io.py: parent -> tests/, parent.parent -> exercise root.

14. Path.unlink()

```
test_file.unlink(missing_ok=True)
```

Deletes a file. missing_ok=True avoids failure if it is already absent.

15. __init__.py

Here it is an empty file allowing tests/ to be imported as a Python package.

```
tests/
|-- __init__.py
|-- test_vertical_state.py
`-- test_file_io.py
```

16. Application code vs test code

Application	Tests
Compute/simulate/save/load	Verify those behaviors
Reusable	Clear and repeatable
Does not know about tests	Imports application code
vertical_state.py / file_io.py	tests/test_*.py

17. main.py

```
main.py -> use the software
tests/  -> verify the software
```

18. run_tests.py

Our educational runner manually executes tests and summarizes outcomes. Frameworks such as pytest automate this in larger projects.

19. One test = one clear behavior

```
test_initial_state()
test_euler_step()
test_missing_file_raises()
```

Specific tests make failures easier to interpret.

20. Repeatability

Running the same test under the same conditions should produce the same result.

21. Independence

A test should not require another test to execute first. Each one arranges its own state.

22. Project structure

```
project/
|-- application modules
|-- main.py
|-- data/
|-- tests/
`-- run_tests.py
```

Structure communicates intent before anyone even opens the files.

23. Cheat sheet

```
assert condition, 'message'

np.allclose(actual, expected)

def test_name():
    # Arrange
    ...
    # Act
    ...
    # Assert
    assert ...

PASS  -> expectations satisfied
FAIL  -> AssertionError
ERROR -> unexpected exception
```